API Timeouts & Deadlines

budgets, propagation, gRPC and cancellation

Watch the 4-min video

In one line: set each timeout a little above the dependency's measured p99 and inside your own SLO, then pass one absolute deadline down the whole call chain so every hop knows how much time is left, fails fast when it runs out, and cancels work nobody is waiting for.

1 The problem: independent timeouts

A user taps Search. The API gateway gives the whole request **2 seconds**. Behind it, the search service calls a ranking service, which calls a feature store. If each hop sets its own independent **10-second** timeout, the user gives up long before the backend does, and your servers keep working on an answer nobody will ever see. That wasted work holds threads, connections and database time that real users need.

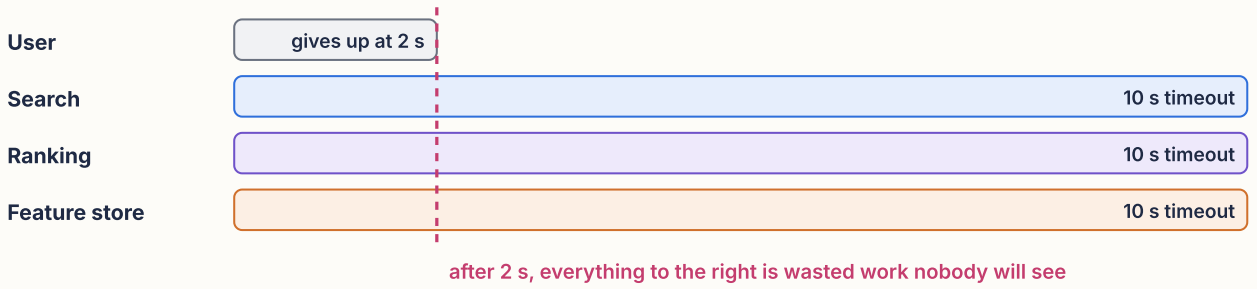
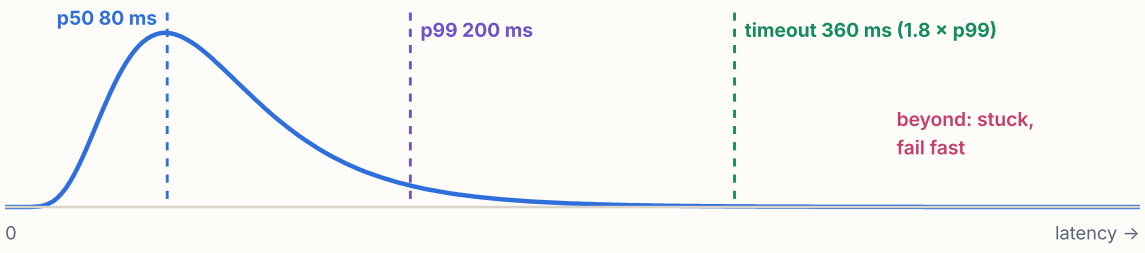


Figure 1. Each timeout looks reasonable on its own. Together they let the backend run five times longer than anyone waits.

2 Choosing a timeout value

Start from **data, not guesses**. Look at the dependency's latency distribution and set the timeout a little above its **p99**: about **1.5 to 2 times the p99**. Normal slow requests still succeed, but truly stuck ones fail fast.

Then check it against your own **SLO**. If your endpoint promises **300 ms at p99**, no dependency can be allowed 2 seconds. And if you plan to retry, all attempts have to fit in the budget too.



normal slow requests (up to p99) still succeed; truly stuck ones are cut off

Figure 2. A timeout set from the distribution. Too tight (near p50) fails healthy requests; too loose (10 s) lets stuck calls pile up.

WORKED EXAMPLE: DOES A RETRY FIT?

Your endpoint's SLO is 300 ms at p99. It calls a pricing service with p99 = 80 ms. A timeout of 1.5 × 80 = **120 ms** leaves 180 ms. One retry after a short jittered pause (say 20 ms) costs at most 120 + 20 + 120 = **260 ms**, leaving 40 ms for your own work. Tight, but it fits.

If pricing's p99 were 150 ms, a 225 ms timeout leaves no room for a retry. Either drop the retry, or ask whether the SLO and the dependency are compatible at all.

3 Think in budgets

If the gateway's total budget is 2 seconds, every downstream call has to fit inside **what's left**. A child call should **never get a timeout longer than its parent's remaining time**. And retries are part of the same budget: three attempts at 1 s each cannot fit in 2 s.

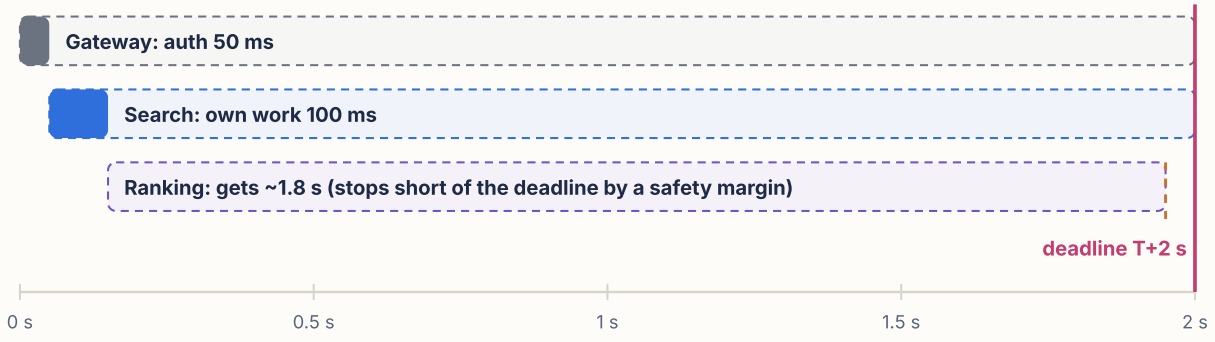
```
child timeout = min(own timeout, parent's remaining time − safety margin)
```

4 Deadline propagation

Instead of passing a *duration* (“you have 10 s”), pass an **absolute deadline**: the moment the answer becomes useless (“stop at 12:00:02.000”). Every hop then computes its remaining time the same way, however long it waited in queues.

- **gRPC** has this built in. The client sets a deadline, it travels with the call, and each server can ask how much time remains (`context.time_remaining()` in Python) and give a smaller deadline to its own calls. When it passes, the call fails with **DEADLINE_EXCEEDED**.
- **Over plain HTTP**, use a header, such as a request deadline or remaining milliseconds, and have every service honor it.

The example: the gateway receives a request at **T = 0** with a 2 s deadline. It spends **50 ms** on auth, then calls search with **1.95 s** remaining. Search spends **100 ms**, then calls ranking with about **1.8 s** remaining, minus a small safety margin. If ranking sees less than, say, **50 ms** left, it doesn't even start: it fails fast with deadline exceeded.



solid = time spent · dashed = time still allowed; each hop forwards the same absolute deadline

Figure 3. One deadline, many hops. Each service spends part of the budget and hands the rest down; nothing runs past T + 2 s.

5 Cancellation, and failing clearly

When the caller gives up, the work below should **stop**.

- In **gRPC**, cancellation propagates automatically to downstream calls.

- In **Python async** code, a timeout (`asyncio.timeout(...)`) cancels the task, so make sure your HTTP calls and database queries are awaited inside it and are cancelled with it.
- Set a **statement timeout** on the database (in Postgres, `SET statement_timeout = '1500ms'`), so an abandoned query doesn't keep burning a connection.



Figure 4. Cancellation should travel all the way down. The database timeout catches anything the layers above failed to cancel.

When time does run out, say so clearly: return **504 Gateway Timeout** at the edge, or **DEADLINE_EXCEEDED** in gRPC, not a vague 500, and **log which hop ran out of time**.

COMMON MISTAKES

- No timeout at all: many HTTP clients wait forever by default.
- Inner timeouts longer than outer ones.
- Retrying after the deadline has already passed.
- Timing out the caller but leaving the downstream query running.
- Returning a vague 500 instead of 504 / DEADLINE_EXCEEDED.

6 Key terms

p99: latency 99% of requests beat

Budget: total time a request may spend

DEADLINE_EXCEEDED: gRPC status when the deadline passes

SLO: target the endpoint promises, e.g. 300 ms p99

Deadline: absolute moment the answer becomes useless

statement_timeout: Postgres limit on one query's run time

7 Interview questions

Q1. How do you choose a timeout value? BASIC

From measured latency: about 1.5–2× the dependency's p99, within your own SLO, and within the caller's remaining budget, leaving room for any retries. A dependency with p99 = 200 ms gets roughly 300–400 ms.

Q2. What is deadline propagation? BASIC

Passing an absolute deadline down the call chain (a gRPC deadline or a deadline header) so every service knows how much time remains, gives its children less, fails fast when time is up, and cancels work nobody needs any more.

Q3. Why an absolute deadline instead of a fixed duration per hop? BASIC

A fixed 10 s per hop ignores time already spent upstream. With an absolute deadline every hop computes the true remaining time, so the chain as a whole never exceeds the user's budget.

Q4. Your endpoint's SLO is 300 ms p99 and a dependency has a 2 s timeout. What's wrong? SCENARIO

That one call alone can take 6× the whole SLO. Reduce it to fit, e.g. 1.5× the dependency's p99, and make sure it is also capped by the request's remaining time.

Q5. Gateway budget 2 s, auth takes 50 ms, search takes 100 ms. What does ranking get? SCENARIO

About 2.0 − 0.05 − 0.1 = 1.85 s, minus a safety margin for network and response time, so roughly 1.8 s. If ranking sees under ~50 ms left, it returns DEADLINE_EXCEEDED at once.

Q6. Users time out but database CPU stays high for minutes afterwards. Why? SCENARIO

The work isn't cancelled: queries keep running after the caller left. Propagate cancellation (gRPC, asyncio task cancellation), and set a statement_timeout so abandoned queries are killed.

Q7. Over HTTP between machines, what is risky about sending an absolute timestamp? TRICKY

Clock skew: if the servers' clocks differ by 300 ms, the budget is wrong by 300 ms. Sending the remaining milliseconds, which each receiver turns into a local deadline, avoids that; gRPC sends the remaining timeout on the wire for this reason.

Q8. What should a service return when the deadline is exceeded? TRICKY

504 Gateway Timeout at the HTTP edge or DEADLINE_EXCEEDED in gRPC, not 500, and log which hop ran out of time. Don't retry once the deadline has passed: nobody is waiting for the answer.

Next: SD-C-020 · Webhooks backendroadmap.com